

深度学习

Lab11-Retrieval task

兰韵诗

这是期末大作业，请在6月14号结束之前提交！

Document Retrieval

- 在现代信息时代，人们面临着海量的信息，如何高效地从中检索到与自己需求最相关的信息成为了一个重要挑战。
- 为了提高信息检索系统的能力，研究人员和工程师们不断尝试各种方法，其中包括改进检索算法、优化搜索引擎等。其中，通过高效的检索方法来召回与用户提出的问题或需求最相关的文档是一个重要的研究方向。



Document Retrieval

- 在给定问题的情况下，通过高效的检索方法，召回与提问最相关的3个文档，提高检索能力。
- 评估指标：Recall（召回率）是指检索系统能够检索到相关文档的能力，而MRR（Mean Reciprocal Rank，平均倒数排名）则是一种衡量检索系统排名效果的指标，它考虑了检索结果中相关文档的排名情况。
- 输入：document.json
- 输出：result.json

2

Dataset

数据集	介绍	条数
document.json	文档 数据集	26599条
query_trainset.json	问题 训练集	2044条
query_testset.json	问题 测试集	512条

数据集介绍——document.json

```
[  
  {  
    "title_input_list" : xxx,      # 标题 经过分词后得到的 token IDs  
    "title_embedding":xxx,        # 标题 embedding向量 , 大小为(N,) , N为embedding的维度。  
    "fact_input_list":xxx,         # 文档 经过分词后得到的 token IDs  
    "facts_embedding":xxx,         # 文档 embedding向量 , 大小为(N,) , N为embedding的维度  
    "source":xxx,                  # 数据来源  
    "category":xxx,                # 数据类型  
    "published_at":xxx             # 发布时间  
  },  
  ....  
]
```

数据集介绍——query_trainset.json

```
[
  {
    "query_input_list": xxx,      # 问题 经过分词后得到的 token IDs
    "query_embedding":xxx,      # 问题 embedding向量 , 大小为(N,) , N为embedding的维度。
    "evidence_list": [
      {
        "fact_input_list":xxx,    # 文档 经过分词后得到的 token IDs
        "fact_embedding":xxx     # 文档 embedding向量
      },
      {
        "fact_input_list":xxx,    # 文档 经过分词后得到的 token IDs
        "fact_embedding":xxx     # 文档 embedding向量
      },
      ...
    ]
  },
  ...
]
```

数据集介绍——query_testset.json

```
[
  {
    "query_input_list":xxx,      # 问题 经过分词后得到的 token IDs
    "query_embedding":xxx,      # 问题 embedding向量，大小为(N,)，N为embedding的维度
    "evidence_list": []         # 空列表
  },
  ....
]
```

返回结果——result.json :

```
[
  {
    "query_input_list" : xxx,          # 问题经过分词后得到的 token IDs
    "evidence_list": [
      { "fact_input_list" : xxx },    # 召回的第一个文档经过分词后得到的 token IDs
      { "fact_input_list" : xxx },    # 召回的第二个文档经过分词后得到的 token IDs
      { "fact_input_list" : xxx }     # 召回的第三个文档经过分词后得到的 token IDs
    ]
  },
  ....
]
```

3

Baseline Model

Cosine_similarity

1. 读取问题和文档 embeddings 向量
2. 对每个问题进行检索：
 - 遍历每个问题，计算其与文档 embeddings 的相似度，并获取前 3 个最相关的文档索引。然后构建结果字典，包括 问题输入列表 和 相关证据列表（包括问题最相关的文档信息），并将结果添加到结果列表中。
3. 将结果写入 JSON 文件：
 - 将结果列表写入到结果文件 output/result.json 中。

4

Evaluation

1. 评分指标

(1) Recall@k

$$recall = \frac{\text{算法结果中相关的}item\text{数量}}{\text{所有相关的}item\text{数量}}$$

表示检索结果中正确答案在前k个返回结果中的比例。

(2) MRR@k

$$MRR = \frac{1}{Q} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

表示平均倒数排名

- 对于每个查询，找到正确答案在返回结果中的排名（如果正确答案存在）。
- 如果正确答案在第m个位置，则MRR@k等于1/m。
- 对所有查询的MRR@k进行平均，即可得到MRR@k。

2. 评分指标

```
def calculate_recall_at_k(result, gold, k):
    result = read_json(result)
    gold = read_json(gold)
    total_queries = len(result)
    recall = 0

    for r, g in zip(result, gold):
        correct_queries = 0
        # 去除开始, 结尾字符
        relevant_facts = [e['fact_input_list'][1:-1] for e in g['evidence_list']]
        top_k_facts = [e['fact_input_list'][1:-1] for e in r['evidence_list'][:k]]
        # 转换成字符串
        relevant_facts = [' '.join(map(str, sublist)) for sublist in relevant_facts]
        top_k_facts = [' '.join(map(str, sublist)) for sublist in top_k_facts]

        if relevant_facts: # 检查relevant_facts是否为空
            for answer in relevant_facts:
                if any(answer in fact for fact in top_k_facts):
                    correct_queries += 1
            recall += correct_queries / len(relevant_facts) # 使用relevant_facts的长度来归一化

    recall_at_k = recall / total_queries
    return recall_at_k
```

```
def calculate_mrr_at_k(result, gold, k):
    result = read_json(result)
    gold = read_json(gold)
    total_queries = len(result)
    mrr = 0
    mrr_at_k = 0

    for r, g in zip(result, gold):
        reciprocal_ranks = []
        relevant_facts = [e['fact_input_list'][1:-1] for e in g['evidence_list']]
        top_k_facts = [e['fact_input_list'][1:-1] for e in r['evidence_list'][:k]]
        relevant_facts = [' '.join(map(str, sublist)) for sublist in relevant_facts]
        top_k_facts = [' '.join(map(str, sublist)) for sublist in top_k_facts]

        if relevant_facts: # 检查relevant_facts是否为空
            for answer in relevant_facts:
                for i, fact in enumerate(top_k_facts):
                    if answer in fact:
                        reciprocal_ranks.append(1 / (i + 1)) # 计算倒数排名并添加到列表中
                        break # 找到第一个匹配的句子后停止搜索
                mrr += sum(reciprocal_ranks) / len(relevant_facts)

    mrr_at_k = mrr / total_queries
    return mrr_at_k
```

- 标准答案里面的fact_input_list只有一句话，只要你检索到的文档里面包含这句话即可。
- evidence_list可能包含0（query无法检索到相关的文档），2，3，4个
- 榜单的分值会作为我们主要打分依据，但我们还会考虑代码的**规范程度**、**方案创新程度**、**代码工作量**对分数(总分100)进行-10 ~ +10分的调整。

3. baseline结果

Phase 1

Phase description

Homework phase1

Max submissions per day: 100

Max submissions total: 100



Download CSV

Download all submissions on leaderboard

Results

#	User	Entries	Time of the Submission	Recall_at_3 ▲	MRR_at_3 ▲
1	51265903028汪小曼	1	2024/05/08 15:17:45	0.2121 (1)	0.1584 (1)

5

Improvement

可以考虑的改进

- 检索后rerank
- 检索算法
-

不限于上述，只是提供参考，大家可以自行选择高效的方法，提高检索的结果

注意事项

- 最多召回 **三个** 最相似的文档
- **只能**使用我们分好的document.json
- 可随意选择模型进行建模、可随意选择python依存包进行建模
- 可选择适合自己的平台跑实验，最后在天梯提交预测数据和代码
- 提交的代码一定要是完整的、可复现的
- 不可以利用额外的数据集，不可以手动标注测试集